

Class 6: Sorting

Reading: Chapter 9, 13
(depends on the book edition)

Linear-time Sorting

Order Statistics



Last Lecture

- ▷ Quick sort – average case $O(n \log n)$
- ▷ The Master theorem
- ▷ Lower Bound for comparison-based sorting algorithms

Faster sorting

- ▷ Can we sort faster than $O(n \log n)$ time?
- ▷ No – if we are limited to comparison-based algorithm
- ▷ So, what else can we do?
 - ▷ Use prior knowledge about the input
 - ▷ Use values not just by comparing them

Counting sort

- ▷ not comparison based
- ▷ input: array $A[1 \dots n]$ of **integers** in $\{1, 2, \dots, k\}$
- ▷ output: array $B[1 \dots n]$: a sorted permutation of A

- ▷ For the solution we will use an auxiliary array $C[1 \dots k]$

Counting sort

example: $n=5$, $k=4$

	1	2	3	4	5
<i>A</i> :	4	1	3	4	3

<i>B</i> :					
------------	--	--	--	--	--

one index for each possible key stored in A			
1	2	3	4
<i>C</i> :			

Initialization

	1	2	3	4	5
A :	4	1	3	4	3

	1	2	3	4
C :	0	0	0	0

B :					
-------	--	--	--	--	--

for $i \leftarrow 1$ **to** k
 do $C[i] \leftarrow 0$

Counting

	1	2	3	4	5
<i>A</i> :	4	1	3	4	3

	1	2	3	4
<i>C</i> :	0	0	0	1

<i>B</i> :					
------------	--	--	--	--	--

for $j \leftarrow 1$ **to** n

do $C[A[j]] \leftarrow C[A[j]] + 1$ $\triangleright C[i] = |\{\text{key} = i\}|$

Counting

A:

1	2	3	4	5
4	1	3	4	3

C:

1	2	3	4
1	0	0	1

B:

--	--	--	--	--

for $j \leftarrow 1$ **to** n

do $C[A[j]] \leftarrow C[A[j]] + 1$ $\triangleright C[i] = |\{\text{key} = i\}|$

Counting

	1	2	3	4	5
A :	4	1	3	4	3

	1	2	3	4
C :	1	0	1	1

B :					
-------	--	--	--	--	--

for $j \leftarrow 1$ **to** n

do $C[A[j]] \leftarrow C[A[j]] + 1 \quad \triangleright C[i] = |\{\text{key} = i\}|$

Counting

	1	2	3	4	5
<i>A</i> :	4	1	3	4	3

	1	2	3	4
<i>C</i> :	1	0	1	2

<i>B</i> :					
------------	--	--	--	--	--

for $j \leftarrow 1$ **to** n

do $C[A[j]] \leftarrow C[A[j]] + 1 \quad \triangleright C[i] = |\{\text{key} = i\}|$

Counting

	1	2	3	4	5
<i>A</i> :	4	1	3	4	3

	1	2	3	4
<i>C</i> :	1	0	2	2

<i>B</i> :					
------------	--	--	--	--	--

for $j \leftarrow 1$ **to** n

do $C[A[j]] \leftarrow C[A[j]] + 1$ $\triangleright C[i] = |\{\text{key} = i\}|$

Counting

A:

1	2	3	4	5
4	1	3	4	3

C:

1	2	3	4
1	0	2	2

B:

--	--	--	--	--

for $j \leftarrow 1$ **to** n

do $C[A[j]] \leftarrow C[A[j]] + 1$ $\triangleright C[i] = |\{\text{key} = i\}|$

Counting

	1	2	3	4	5
<i>A</i> :	4	1	3	4	3

	1	2	3	4
<i>C</i> :	1	0	2	2

<i>B</i> :	1				
------------	---	--	--	--	--

Preparing output

	1	2	3	4	5
<i>A</i> :	4	1	3	4	3

	1	2	3	4
<i>C</i> :	1	0	2	2

<i>B</i> :	1				
------------	---	--	--	--	--

Preparing output

	1	2	3	4	5
<i>A</i> :	4	1	3	4	3

	1	2	3	4
<i>C</i> :	1	0	2	2

<i>B</i> :	1	3	3		
------------	---	---	---	--	--

Preparing output

	1	2	3	4	5
<i>A</i> :	4	1	3	4	3

	1	2	3	4
<i>C</i> :	1	0	2	2

<i>B</i> :	1	3	3	4	4
------------	---	---	---	---	---

B is sorted!

Preparing output

	1	2	3	4	5
<i>A</i> :	4	1	3	4	3

	1	2	3	4
<i>C</i> :	1	0	2	2

<i>B</i> :	1	3	3	4	4
------------	---	---	---	---	---

B is sorted!

What is wrong here?

Preparing output

A:

1	2	3	4	5
4	1	3	4	3

C:

1	2	3	4
1	0	2	2

B:

1	3	3	4	4
---	---	---	---	---

B is sorted!

We only sorted the keys!!

Cumulative counts

	1	2	3	4	5
<i>A</i> :	4	1	3	4	3

	1	2	3	4
<i>C</i> :	1	0	2	2

<i>B</i> :					
------------	--	--	--	--	--

Cumulative counts

	1	2	3	4	5
<i>A</i> :	4	1	3	4	3

	1	2	3	4
<i>C</i> :	1	0	2	2

<i>B</i> :					
------------	--	--	--	--	--

<i>C'</i> :	1	1	2	2
-------------	---	---	---	---

for $i \leftarrow 2$ **to** k

do $C[i] \leftarrow C[i] + C[i-1]$

$\triangleright C[i] = |\{\text{key} \leq i\}|$

Cumulative counts

	1	2	3	4	5
<i>A</i> :	4	1	3	4	3

	1	2	3	4
<i>C</i> :	1	0	2	2

<i>B</i> :					
------------	--	--	--	--	--

<i>C'</i> :	1	1	3	2
-------------	---	---	---	---

for $i \leftarrow 2$ **to** k

do $C[i] \leftarrow C[i] + C[i-1]$ $\triangleright C[i] = |\{\text{key} \leq i\}|$

Cumulative counts

	1	2	3	4	5
<i>A</i> :	4	1	3	4	3

	1	2	3	4
<i>C</i> :	1	0	2	2

<i>B</i> :					
------------	--	--	--	--	--

<i>C'</i> :	1	1	3	5
-------------	---	---	---	---

for $i \leftarrow 2$ **to** k

do $C[i] \leftarrow C[i] + C[i-1]$

$\triangleright C[i] = |\{\text{key} \leq i\}|$

Copy the elements of A

	1	2	3	4	5
<i>A</i> :	4	1	3	4	3

	1	2	3	4
<i>C</i> :	1	1	3	5

<i>B</i> :					
------------	--	--	--	--	--

```
for  $j \leftarrow n$  downto 1  
  do  $B[C[A[j]]] \leftarrow A[j]$   
      $C[A[j]] \leftarrow C[A[j]] - 1$ 
```

Copy the elements of A

	1	2	3	4	5
<i>A</i> :	4	1	3	4	3

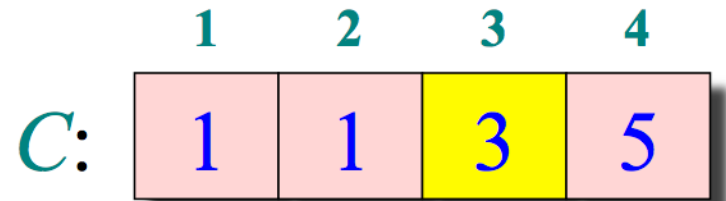
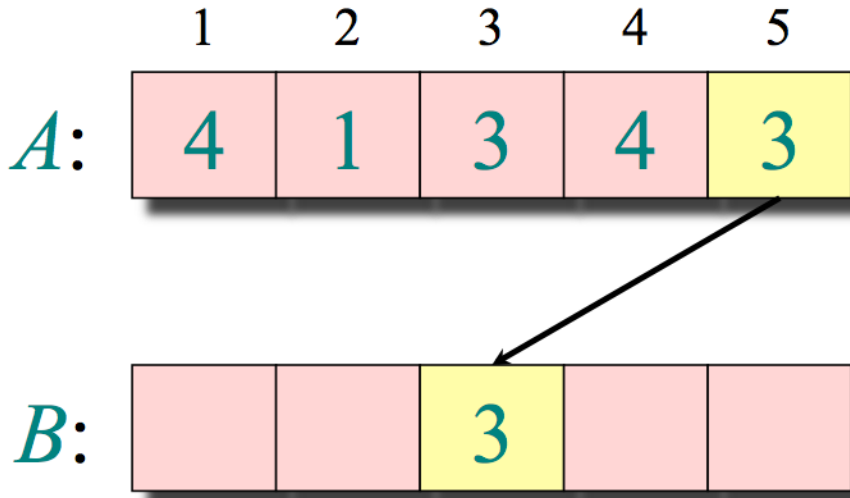
	1	2	3	4
<i>C</i> :	1	1	3	5

<i>B</i> :					
------------	--	--	--	--	--

*There are exactly 3 elements $\leq A[5]$;
so where should I place $A[5]$?*

```
for  $j \leftarrow n$  downto 1  
  do  $B[C[A[j]]] \leftarrow A[j]$   
       $C[A[j]] \leftarrow C[A[j]] - 1$ 
```

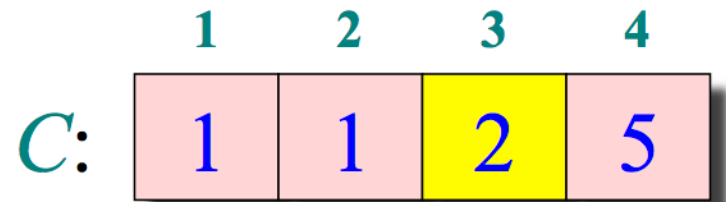
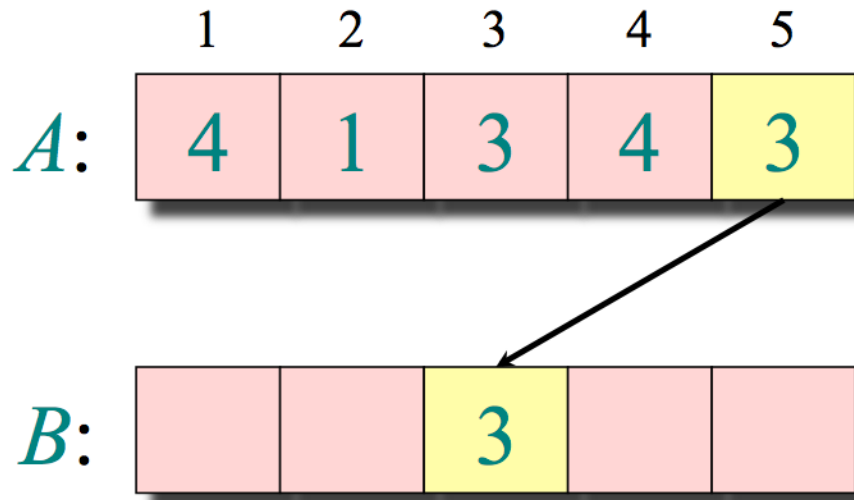

Copy the elements of A



Used-up one 3; update counter.

```
for  $j \leftarrow n$  downto 1  
  do  $B[C[A[j]]] \leftarrow A[j]$   
      $C[A[j]] \leftarrow C[A[j]] - 1$ 
```

Copy the elements of A



```
for  $j \leftarrow n$  downto 1  
  do  $B[C[A[j]]] \leftarrow A[j]$   
      $C[A[j]] \leftarrow C[A[j]] - 1$ 
```

Copy the elements of A

	1	2	3	4	5
<i>A</i> :	4	1	3	4	3

	1	2	3	4
<i>C</i> :	1	1	2	5

<i>B</i> :			3		
------------	--	--	---	--	--

```
for  $j \leftarrow n$  downto 1  
  do  $B[C[A[j]]] \leftarrow A[j]$   
      $C[A[j]] \leftarrow C[A[j]] - 1$ 
```

Copy the elements of A

	1	2	3	4	5
<i>A</i> :	4	1	3	4	3

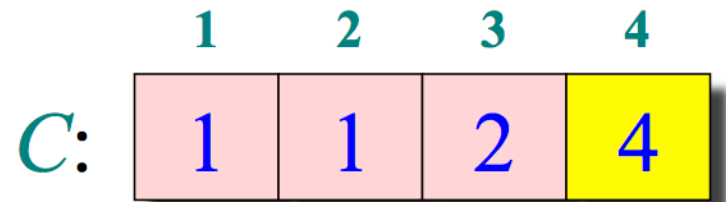
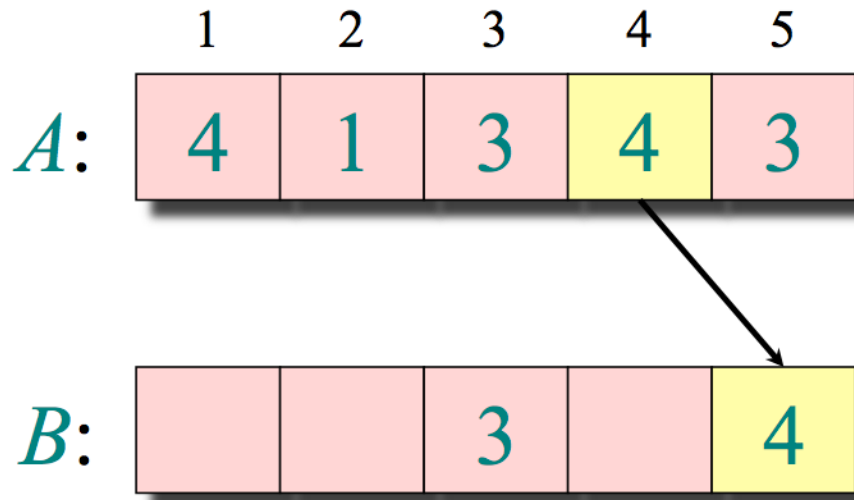
	1	2	3	4
<i>C</i> :	1	1	2	5

<i>B</i> :			3		
------------	--	--	---	--	--

*There are exactly 5 elements $\leq A[4]$,
so where should I place $A[4]$?*

```
for  $j \leftarrow n$  downto 1  
  do  $B[C[A[j]]] \leftarrow A[j]$   
      $C[A[j]] \leftarrow C[A[j]] - 1$ 
```

Copy the elements of A



```
for  $j \leftarrow n$  downto 1  
  do  $B[C[A[j]]] \leftarrow A[j]$   
      $C[A[j]] \leftarrow C[A[j]] - 1$ 
```

Copy the elements of A

	1	2	3	4	5
<i>A</i> :	4	1	3	4	3

	1	2	3	4
<i>C</i> :	1	1	2	4

<i>B</i> :			3		4
------------	--	--	---	--	---

```
for  $j \leftarrow n$  downto 1  
  do  $B[C[A[j]]] \leftarrow A[j]$   
      $C[A[j]] \leftarrow C[A[j]] - 1$ 
```

Copy the elements of A

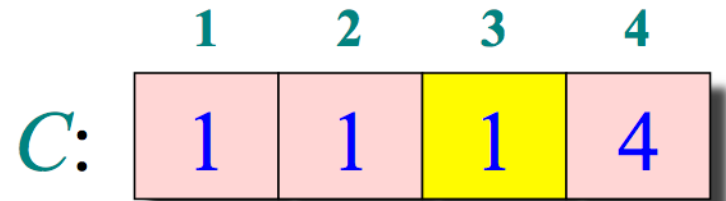
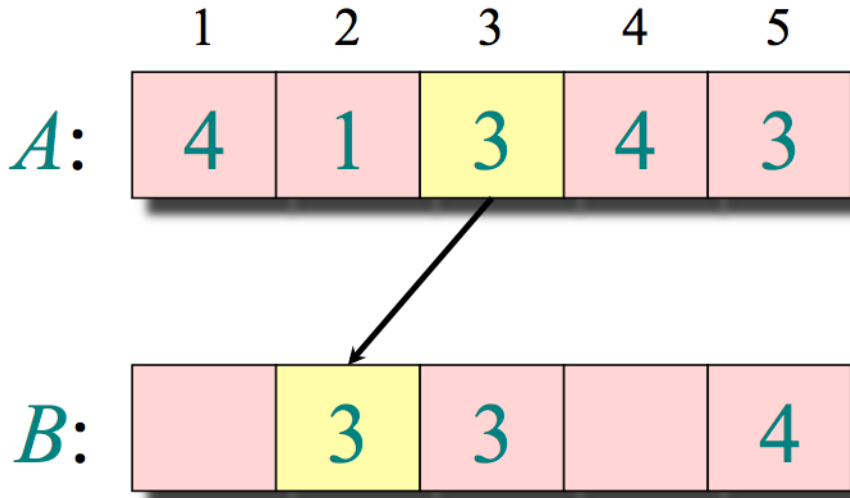
	1	2	3	4	5
<i>A</i> :	4	1	3	4	3

	1	2	3	4
<i>C</i> :	1	1	2	4

<i>B</i> :			3		4
------------	--	--	---	--	---

```
for  $j \leftarrow n$  downto 1  
  do  $B[C[A[j]]] \leftarrow A[j]$   
      $C[A[j]] \leftarrow C[A[j]] - 1$ 
```

Copy the elements of A



```
for  $j \leftarrow n$  downto 1  
  do  $B[C[A[j]]] \leftarrow A[j]$   
      $C[A[j]] \leftarrow C[A[j]] - 1$ 
```


Copy the elements of A

	1	2	3	4	5
<i>A</i> :	4	1	3	4	3

	1	2	3	4
<i>C</i> :	1	1	1	4

<i>B</i> :		3	3		4
------------	--	---	---	--	---

```
for  $j \leftarrow n$  downto 1  
  do  $B[C[A[j]]] \leftarrow A[j]$   
       $C[A[j]] \leftarrow C[A[j]] - 1$ 
```

Copy the elements of A

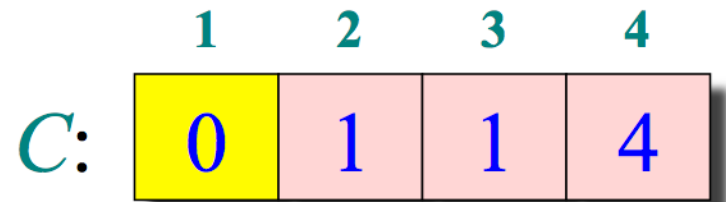
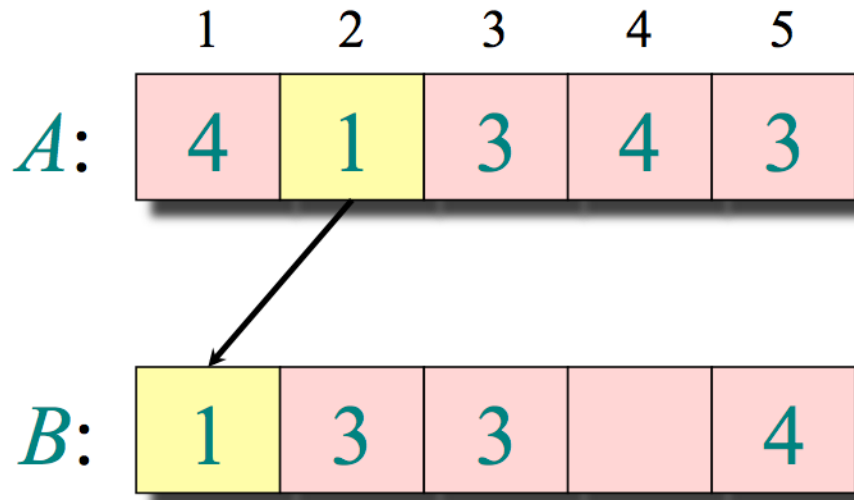
	1	2	3	4	5
<i>A</i> :	4	1	3	4	3

	1	2	3	4
<i>C</i> :	1	1	1	4

<i>B</i> :		3	3		4
------------	--	---	---	--	---

```
for  $j \leftarrow n$  downto 1
  do  $B[C[A[j]]] \leftarrow A[j]$ 
      $C[A[j]] \leftarrow C[A[j]] - 1$ 
```

Copy the elements of A



```
for  $j \leftarrow n$  downto 1  
  do  $B[C[A[j]]] \leftarrow A[j]$   
       $C[A[j]] \leftarrow C[A[j]] - 1$ 
```

Copy the elements of A

	1	2	3	4	5
<i>A</i> :	4	1	3	4	3

	1	2	3	4
<i>C</i> :	0	1	1	4

<i>B</i> :	1	3	3		4
------------	---	---	---	--	---

```
for  $j \leftarrow n$  downto 1  
  do  $B[C[A[j]]] \leftarrow A[j]$   
       $C[A[j]] \leftarrow C[A[j]] - 1$ 
```

Copy the elements of A

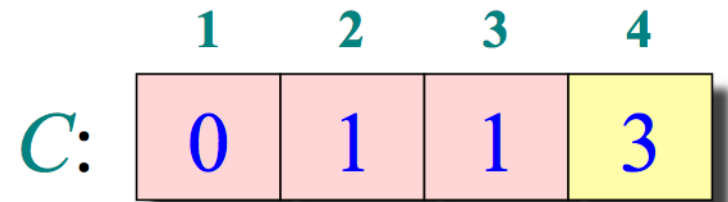
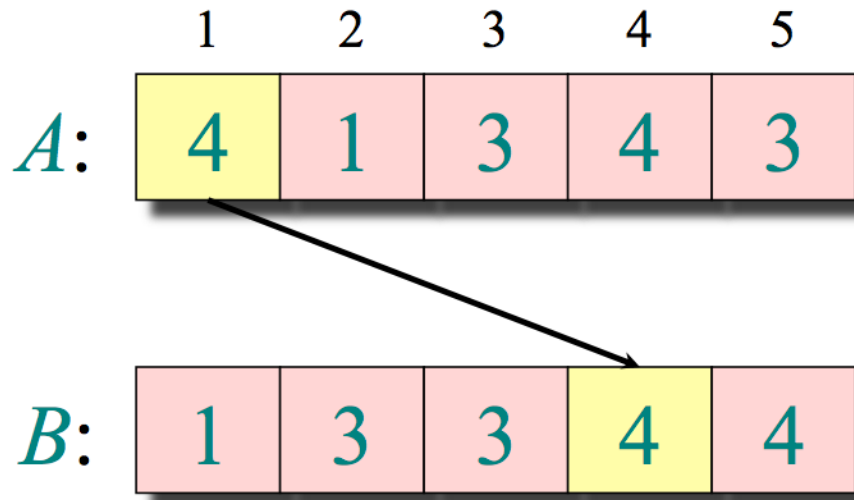
	1	2	3	4	5
<i>A</i> :	4	1	3	4	3

	1	2	3	4
<i>C</i> :	0	1	1	4

<i>B</i> :	1	3	3		4
------------	---	---	---	--	---

```
for  $j \leftarrow n$  downto 1  
  do  $B[C[A[j]]] \leftarrow A[j]$   
       $C[A[j]] \leftarrow C[A[j]] - 1$ 
```

Copy the elements of A



Stable Sort!

```
for  $j \leftarrow n$  downto 1  
  do  $B[C[A[j]]] \leftarrow A[j]$   
      $C[A[j]] \leftarrow C[A[j]] - 1$ 
```

Analysis

CountingSort(A, n, k)

$\Theta(k)$

for $i \leftarrow 1$ **to** k
 do $C[i] \leftarrow 0$

$\Theta(n)$

for $j \leftarrow 1$ **to** n
 do $C[A[j]] \leftarrow C[A[j]] + 1$

$\Theta(k)$

for $i \leftarrow 2$ **to** k
 do $C[i] \leftarrow C[i] + C[i-1]$

$\Theta(n)$

for $j \leftarrow n$ **downto** 1
 do $B[C[A[j]]] \leftarrow A[j]$
 $C[A[j]] \leftarrow C[A[j]] - 1$

$\Theta(n + k)$

so $\Theta(n)$ if $k = O(n)$

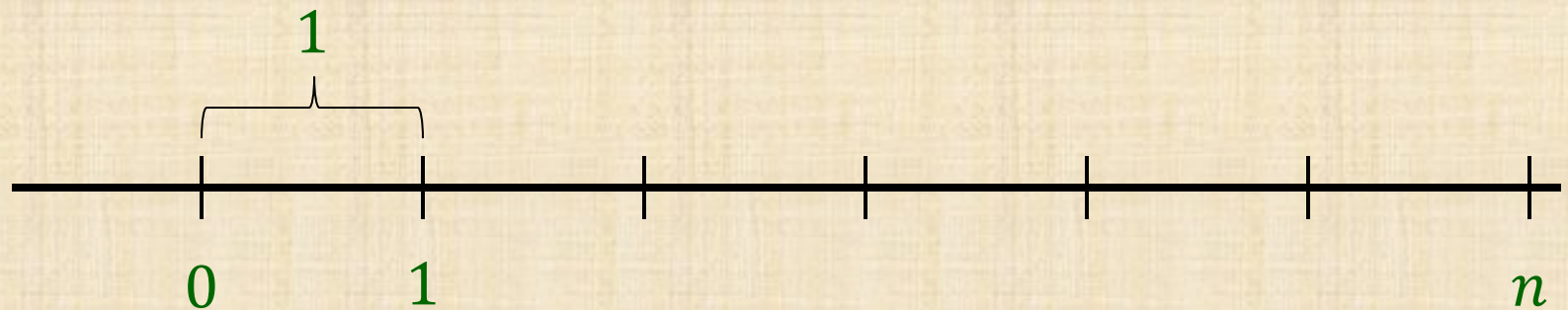
Bucket Sort

- ▷ What about sorting real numbers?
- ▷ Assumptions:
 - ▷ n elements
 - ▷ All keys are numbers in $[0, n)$ possibly not integers
 - ▷ Keys are distributed uniformly in the interval



Bucket Sort

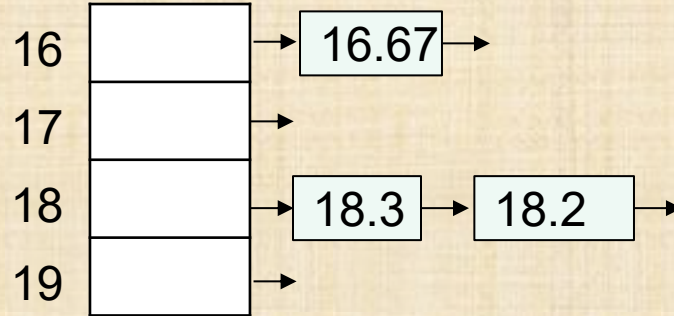
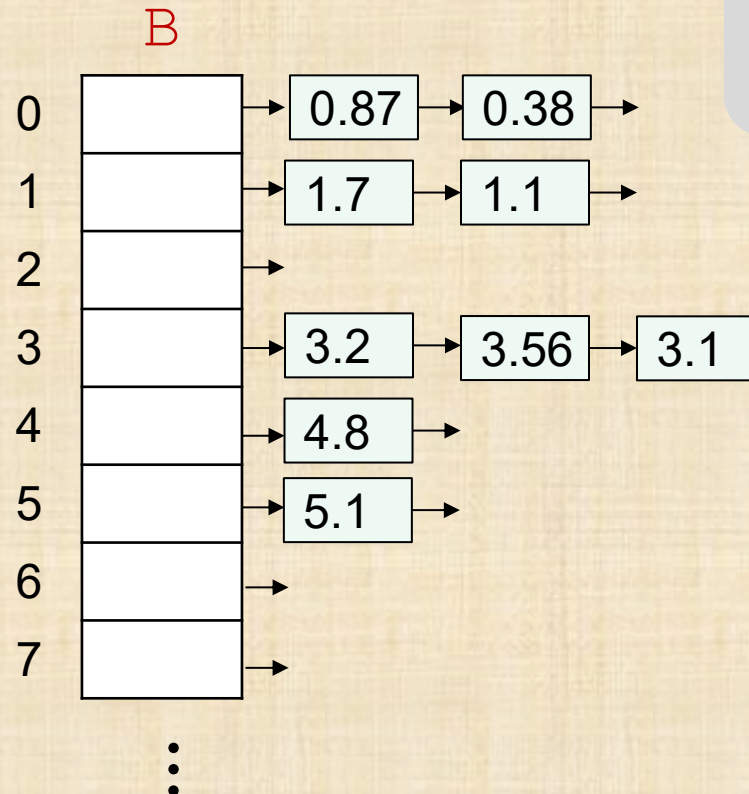
- ▷ Split the interval $[0, n)$ into n sub-intervals (buckets) of size 1 each
- ▷ Place each element in its bucket
- ▷ Maintain the elements in each bucket in a linked list



$n=20$
 $Bucket(k)=\lfloor k \rfloor$

	A
0	1.7
1	3.2
2	3.56
3	18.3
4	0.87
5	1.1
6	4.8
7	3.1
⋮	

16	18.2
17	5.1
18	0.38
19	16.67



A: input array
B: Array of lists

BucketSort(A)

$n \leftarrow \text{length}(A)$

$B \leftarrow$ array of n empty linked lists

for $i \leftarrow 0$ **to** $n-1$

do *insert* $A[i]$ into list $B[A[i]]$

for $j \leftarrow 0$ **to** $n-1$

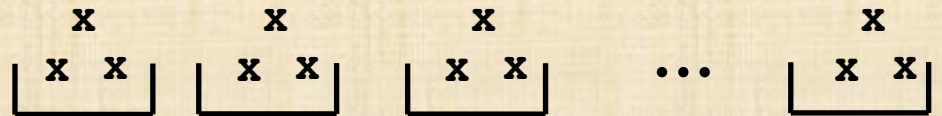
do SortList $B[j]$ //say, Bubble sort

Concatenate the lists $B[0], B[1], \dots, B[n-1]$

Bucket Sort: Running Time

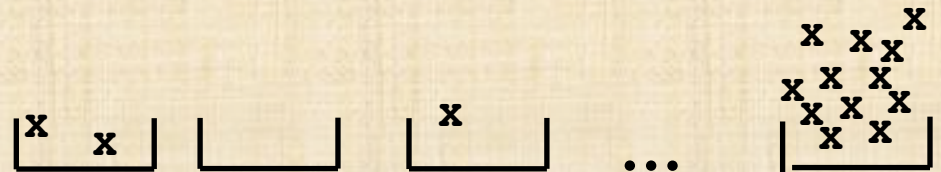
- ▷ $O(n)$ to distribute elements to buckets
- ▷ $O(n)$ to concatenate the sorted buckets
- ▷ How many steps are required to sort all buckets?
 - ▷ Best case: There is a constant c such that each bucket contains no more than c keys:

n times $\theta(1) = \theta(n)$



- ▷ Worst case: some bucket has $\theta(n)$ keys:

$\theta(n^2)$



Bucket Sort: Running Time

- ▷ $O(n)$ to distribute elements to buckets
- ▷ Uniformly distributed keys
 - ▷ the expected # of keys per bucket = $O(1)$
 - ▷ time to sort a bucket with k elements is $O(k^2)$
 - ▷ It can be shown: the expected sum of squares of bucket sizes is $O(n)$ (proof omitted)
- ▷ $O(n)$ to concatenate the sorted buckets

Bucket Sort: Running Time

- ▷ Best and average case $\Theta(n)$
- ▷ Worst case $\Theta(n^2)$
 - ▷ When?
 - ▷ Can we make this $\Theta(n \log n)$?

General Bucket Sort

- ▷ The restriction that the range of keys is $[0, n)$ is not essential
- ▷ If keys are numbers in $[l, h)$ rescale to $[0, n)$.
- ▷ Equivalently, divide into buckets of size $\frac{h-l}{n}$
- ▷ Can also be used for any other known distributions:
 - ▷ Height of people
 - ▷ Age of students
 - ▷ Driving speed
 - ▷ ...

Radix-Sort

A: input array
d: number of
fields

```
RadixSort (A, d)
```

```
  for i ← 1 to d
```

```
    do a linear-stable sort of A,  
        according to field i
```

- ▷ Let $T_{ss}(n) = O(n)$ be the complexity of the stable sorting algorithm used

Running Time: $O(dT_{ss}(n)) = O(dn)$

$$T_{ss} = T_{\text{stable sort}}$$

Digits Radix Sort: $d=3$

329

457

657

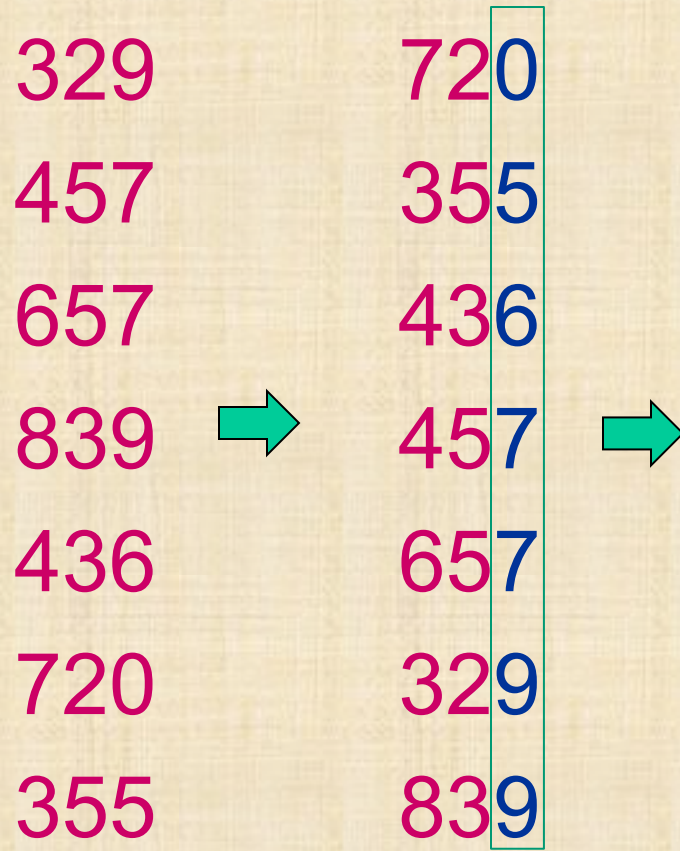
839 →

436

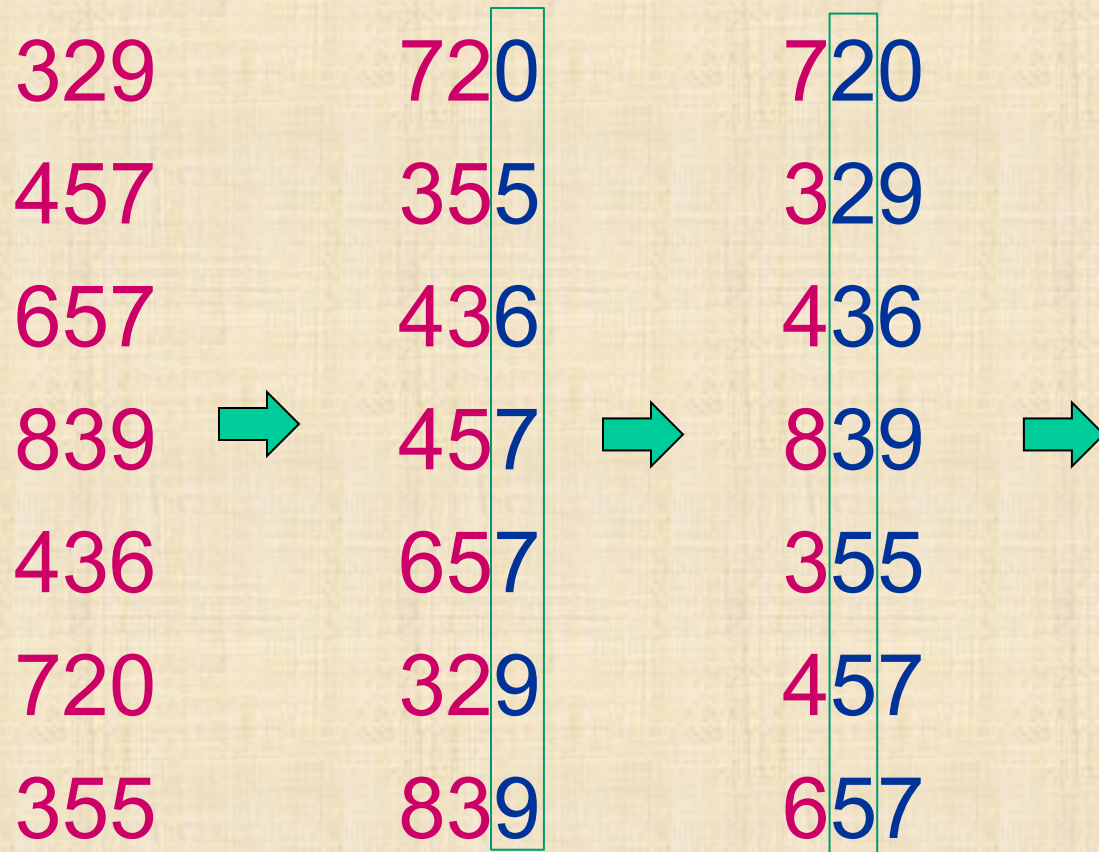
720

355

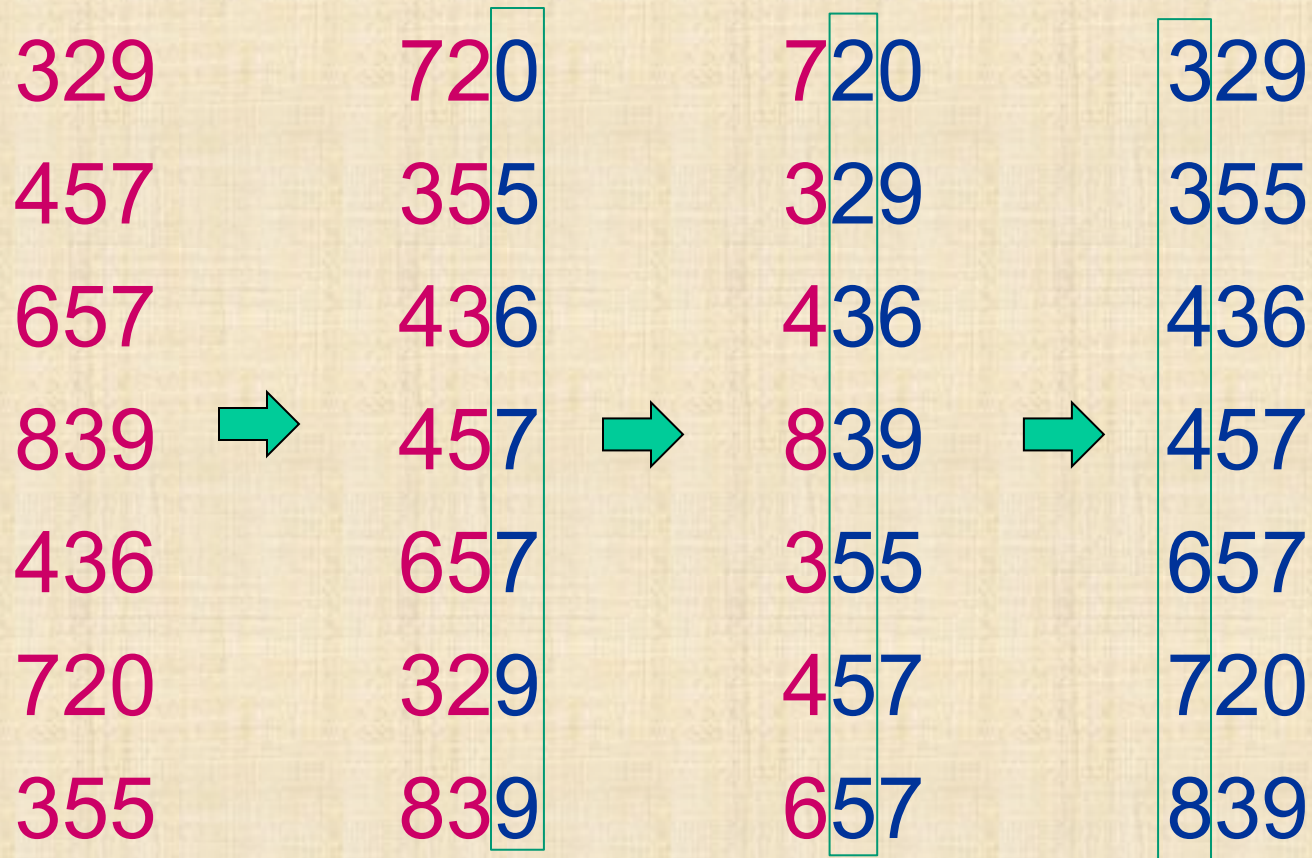
Digits Radix Sort: d=3



Digits Radix Sort: $d=3$



Digits Radix Sort: $d=3$



Lexicographic Order of Words

Most
significant?

Least
significant?

▷ Let

$$x = (x_d x_{d-1} \dots x_1) \quad y = (y_d y_{d-1} \dots y_1)$$

▷ $x < y \Leftrightarrow$ if for some t ,

$$x_t < y_t \text{ \& } x_k = y_k \text{ for all } k > t$$

- ▷ All words should have the same length: fill in shorter word with trailing blanks
- ▷ The character '_' (blank) is smaller than all characters.

▷ Example:

$$\text{safe_} < \text{sail_} < \text{salt_} < \text{salty}$$

General Radix Sort

- ▷ Key can have fields of different types

$f_d, f_{d-1}, \dots, f_1$

- ▷ e.g. date = year, month, day

- ▷ Use a **stable** sort:

- ▷ First sort according to f_1 = least significant “digit”

- ▷ Then sort according to on f_2 = next significant “digit”

- ▷ ...

- ▷ Note the importance of sorting in this order, using a stable sort

Radix Sort - summary

- ▷ Assumption:
 - ▷ Each key consist of d fields.
 - ▷ A stable $O(n)$ -time sort is applicable to each field (e.g., counting sort)
- ▷ Time complexity: $\Theta(dn)$
- ▷ If the number of fields (digits) is constant $d = O(1)$ then running time is $\Theta(n)$
- ▷ In particular, can sort n integers in $[1, n^d]$ in $\Theta(dn)$ time (see recitation)

Summary of Sorting

- ▷ Worst case and average case running time of **any** comparison-based sorting algorithm are $\Omega(n \log n)$
- ▷ Examples of sorting algorithms with worst case running time $\Theta(n)$
 - ▷ Not comparison based!

List of Sorting Algorithms

- ▷ Linear-time: CountingSort, BucketSort, RadixSort
- ▷ Comparison based:
 - ▷ BubbleSort and InsertionSort
 - ▷ HeapSort
 - ▷ MergeSort
 - ▷ QuickSort

Order Statistics

ORDER STATISTICS: Who is the i -th **SMALLEST** element? 🤔

Let's find out with our data-animals!

Hey! I'm $X(4)$!
The 4th element in order!

SORTED Order
(from Smallest to Largest)

i

Our sorted sample:
1, 2, 3, **5**, 8, 13...

DEFINITION: For a sorted list $X(1) \leq X(2) \leq \dots \leq X(n)$,
the i -th **Order Statistic** is the value $X(i)$.

Wait, I want a better spot!
Am I the smallest?

Nope, I'm $X(10)$!
The biggest!

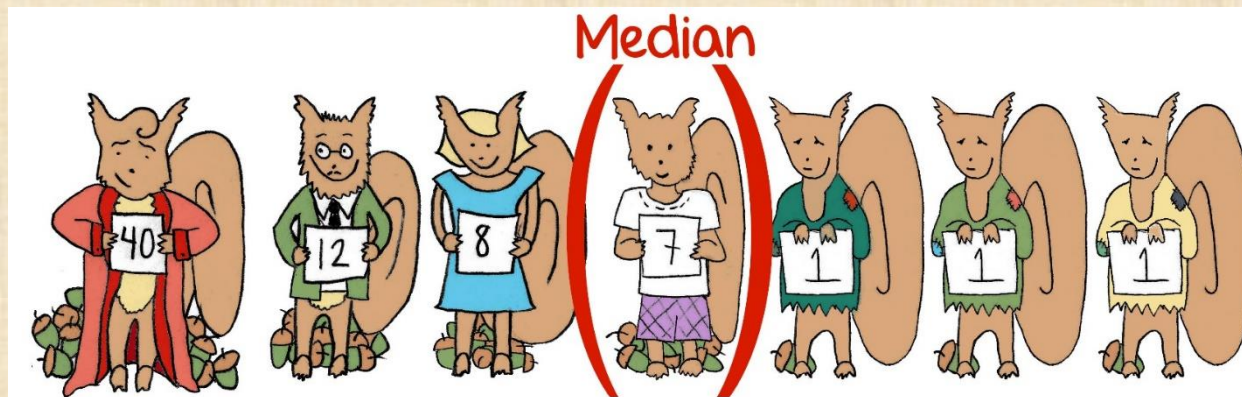
Order Statistics

Definition: The *rank* of a value x in a set S is the location of x in a sorted list of the elements of S

Definition: the i^{th} order statistic in a set S is the i^{th} smallest element

The i^{th} order statistic is the element with rank i .

Example: the rank of 5 in $\{8, 2, 7, 5\}$ is 2



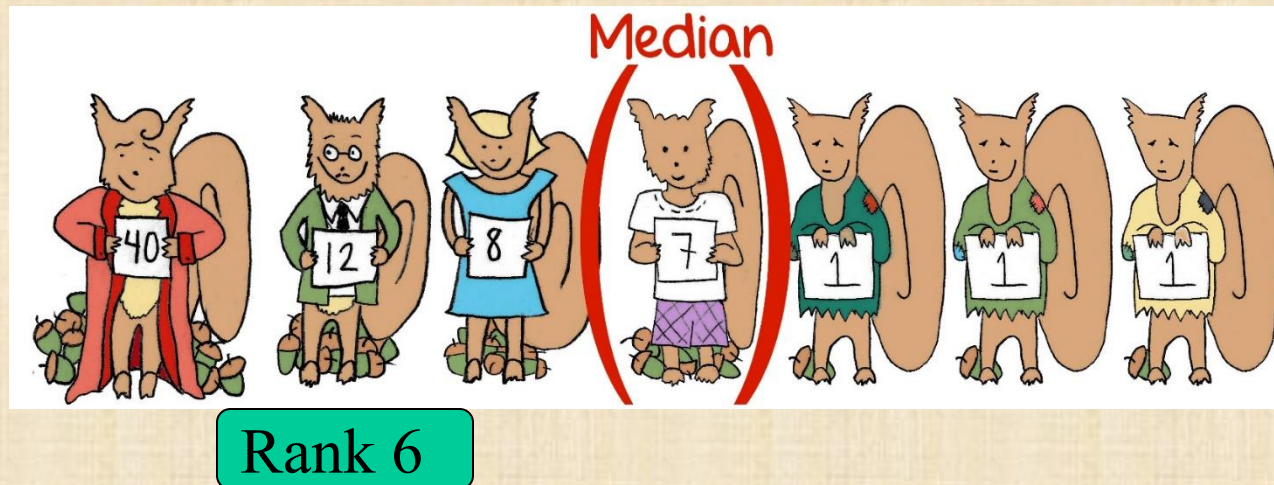
Order Statistics

Let S be a set with n elements

- ▷ The minimum of S has rank = 1
- ▷ The maximum of S has rank = n
- ▷ The median of S
 - ▷ If n is odd, the median has rank $(n + 1)/2$
 - ▷ If n is even, there are two medians having ranks $\lceil (n + 1)/2 \rceil$ and $\lfloor (n + 1)/2 \rfloor$

Order Statistics

- ▷ Given an unsorted array:
 - ▷ We want to find the median
 - ▷ We want to find the rank of the key **12**
- ▷ Can we avoid sorting?



The Selection Problem

- ▷ *Select*(A, k): find an element of rank k in A
- ▷ We can sort and answer, in $O(n \log n)$
- ▷ Can we do better?

Finding Min (or Max)

Minimum(A)

min $\leftarrow$ A[1]

for i $\leftarrow$ 2 to length(A) do

 if min > A[i] then

 min $\leftarrow$ A[i]

return min

- ▷ What is the running time?
- ▷ Can we do better?

Straightforward Method

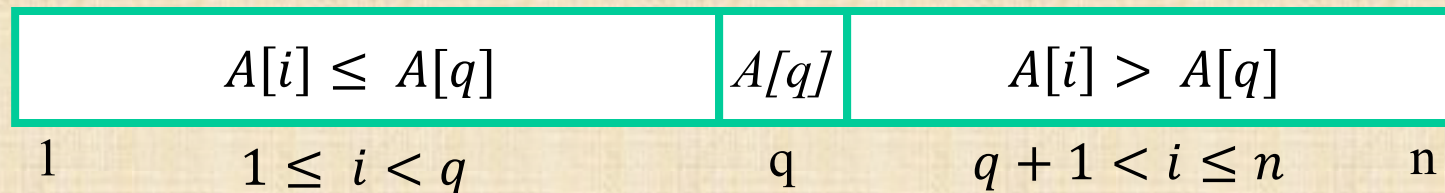
- ▷ *Select*(A, k)
 - ▷ Find the minimum and discard it, k times
- ▷ Complexity: $\theta(nk)$
 - ▷ For $k = \Omega(n)$, time is $\theta(n^2)$
 - ▷ Bad for large values of k
 - ▷ Bad for finding the median
- ▷ Can we do better?

Quick Selection

The task: find an element of rank k in A

Key idea:

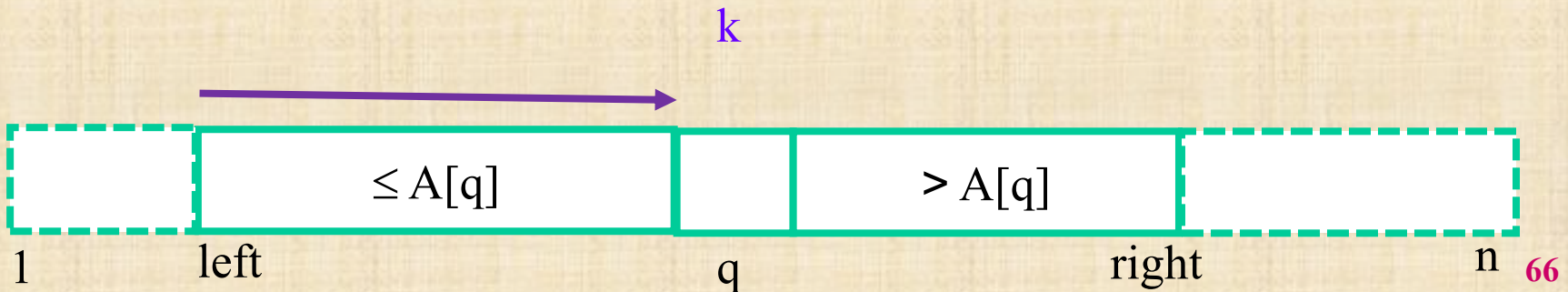
- ▷ Partition the array as in Quick-sort.
- ▷ Let q be the pivot location in A after the partition
- ▷ If $q = k$, return $A[q]$
- ▷ If $q \neq k$, recursively search in the appropriate side for the element of $rank(k)$



A: input array
left, right: indexes
k: rank in A

Quick Selection

```
QuickSelect(A, left, right, k)
    if (left == right) then return A[left];
    q = Partition(A, left, right)
    if (q = k) then return A[q];
    if (k < q) then
        return QuickSelect(A, left, q-1, k);
    else
        return QuickSelect(A, q+1, right, k);
```



Example Calls for Quick Select

- ▷ Find median element:
 - ▷ QuickSelect(A, 1, n, n/2)
- ▷ Find the element of rank(k):
 - ▷ QuickSelect(A, 1, n, k)
- ▷ Find the element of rank(6):
QuickSelect(A, 1, 10, 6)

A	17	6	34	18	9	5	19	22	28	11
	1	2	3	4	5	6	7	8	9	10

Analyzing Quick Selection

- ▷ Worst case: partition always $0 : n-1$

$$\begin{aligned} T(n) &= T(n-1) + \Theta(n) \\ &= \Theta(n^2) \quad (\text{arithmetic series}) \end{aligned}$$

- ▷ No better than sorting!

- ▷ “Best” case: partition always $n/2 : n/2$

$$T(n) = T(n/2) + \Theta(n) = \Theta(n)$$

- ▷ Better than sorting!

Analyzing Quick Selection

▷ Average case:

▷ Let $T(1) = b$

▷ For upper bound, assume the $\text{rank}(i)$ element always falls in larger side of partition:

$$T_{avg}(n) \leq \frac{1}{n} \sum_{q=1}^n T_{avg}(\max(q-1, n-q)) + an$$

partition: $\Theta(n)$



▷ We show that $T_{avg}(n) = O(n)$ by induction

Analyzing Quick Selection

Claim:

Let $T_{avg}(n) \leq \frac{1}{n} \sum_{q=1}^n T_{avg}(\max(q-1, n-q)) + an$, and $T_{avg}(1) = b$. Then $T_{avg}(n) = O(n)$

That is, $\exists c > 0, \exists n_0$, s.t., $\forall n \geq n_0, T_{avg}(n) \leq cn$

Proof (by induction) :

- ▷ Choose $n_0 = 1, c = \max\{b, \dots ?\}$ (? - see next slides)
- ▷ Base case: $T_{avg}(1) = T(1) = b \leq c$
- ▷ Induction hypothesis: $\forall n' < n, T_{avg}(n') < cn'$

Some Algebra

Induction step:

$$\begin{aligned} T_{avg}(n) &\leq \frac{1}{n} \sum_{q=1}^n T_{avg}(\max(q-1, n-q)) + an \\ &= \frac{1}{n} \sum_{q=0}^{n-1} T_{avg}(\max(q, n-q-1)) + an \\ &\leq \frac{2}{n} \sum_{q=\frac{n}{2}}^{n-1} T_{avg}(\max(q, n-q-1)) + an \\ &= \frac{2}{n} \sum_{q=\frac{n}{2}}^{n-1} T_{avg}(q) + an \leq \frac{2}{n} \sum_{q=\frac{n}{2}}^{n-1} cq + an \end{aligned}$$

$$\frac{2}{n} \sum_{q=\frac{n}{2}}^{n-1} cq + an = \frac{\cancel{2c} \left(\frac{n}{2} + (n-1) \right) \cancel{n}}{\cancel{n} \cdot 2} + an$$

$$\leq c \frac{3n-2}{4} + an$$

$$= n \left(\frac{3c}{4} + a \right) - \frac{c}{2}$$

$$\leq n \left(\frac{3c}{4} + \frac{c}{4} \right) - \frac{c}{2} \quad \text{for } c \geq 4a$$

$$\leq cn$$

choose $c = \max\{b, 4a\}$

Rank selection analysis

Time complexity:

- ▷ best / average $\Theta(n)$
- ▷ worst $\Theta(n^2)$

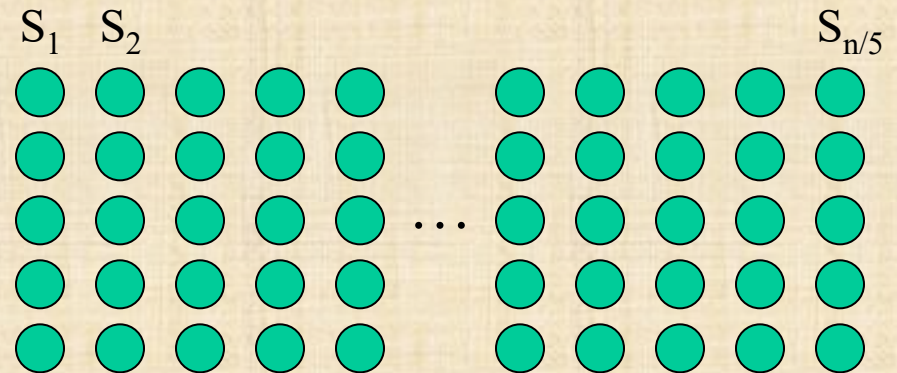
Our next goal: An algorithm whose worst-case time complexity is $\Theta(n)$.

Selection in Linear Time

- ▷ This algorithm is known as:
Median of Medians algorithm
- ▷ A worst-case $O(n)$ time algorithm to find the element with rank k in an unsorted array
- ▷ We will choose a pivot such that at least 30% are larger and at least 30% are smaller than the pivot
- ▷ This guarantees that in the recursive call, the problem size is at most 70% of the current size, which guarantees total linear running time.

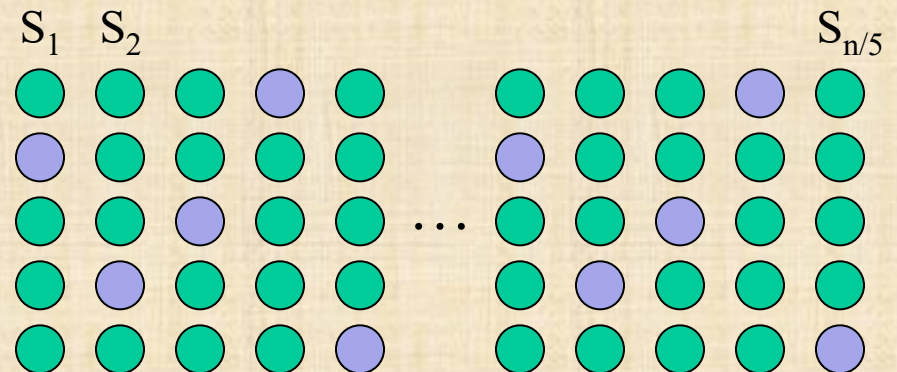
Choosing the pivot

- ▷ Divide the n element into $\lfloor n/5 \rfloor$ groups of 5 elements, S_i .



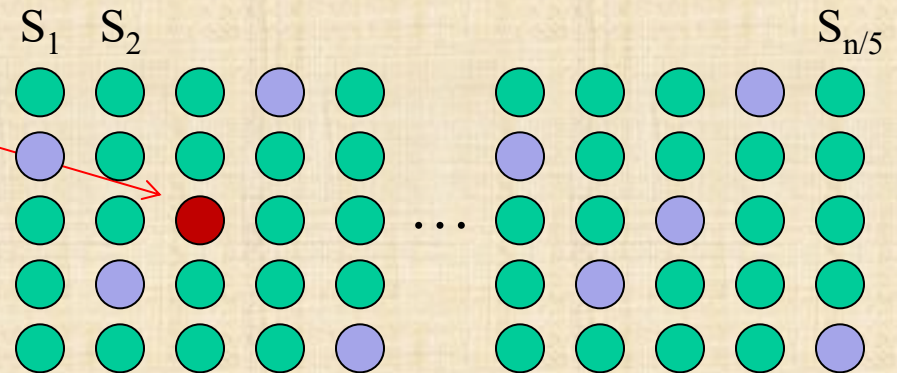
Choosing the pivot

- ▷ Divide the n element into $\lfloor n/5 \rfloor$ groups of 5 elements, S_i .
- ▷ Find the median of each S_i



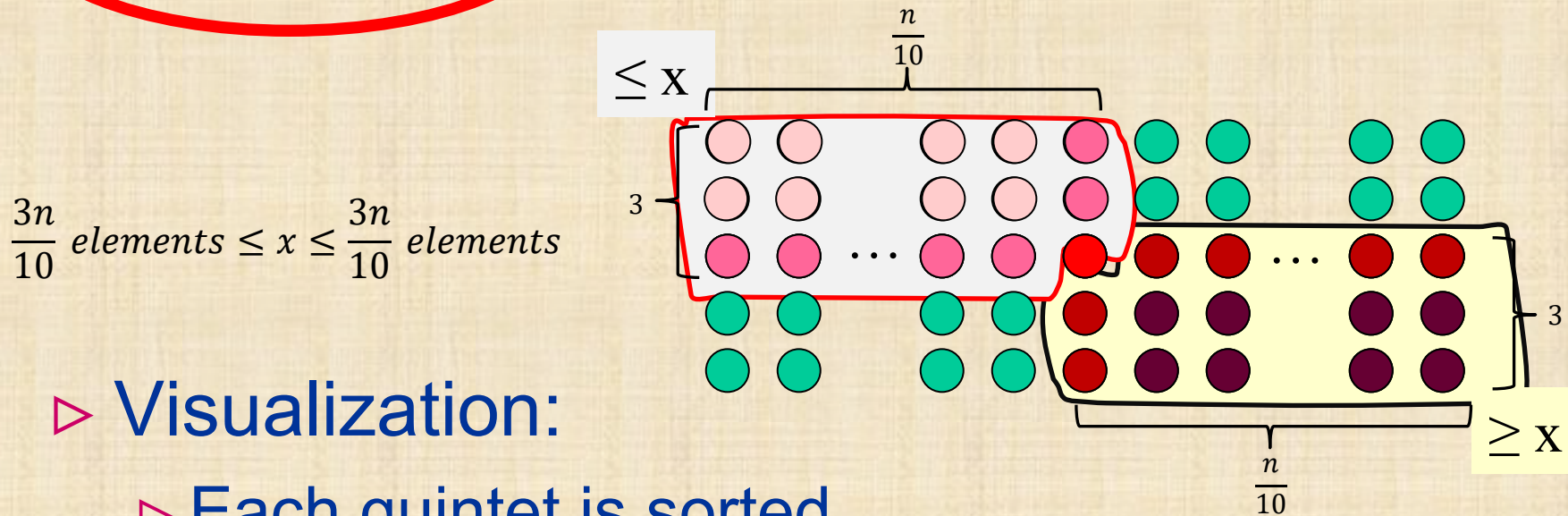
Choosing the pivot

- ▷ Divide the n element into $\lfloor n/5 \rfloor$ groups of 5 elements, S_i .
- ▷ Find the median of each S_i
- ▷ Recursively find the median of the medians, x
- ▷ Partition around x
- ▷ Recursively find the element with rank k accordingly



Choosing the pivot

- ▶ At least 30% of the elements are larger than x
- ▶ At least 30% of the elements are smaller than x



▶ Visualization:

- ▶ Each quintet is sorted
- ▶ The quintets are sorted according to their medians

MM-Select(A,k)

1. If $n < 10$, compute the median naively and return it
2. Divide the n elements into $n/5$ groups S_i of 5 elements each.
3. Find the median s_i of each group S_i naively
4. Recursively call **MM-Select**($\{s_1, \dots, s_{\frac{n}{5}}\}, \frac{n}{5}$) to find x , the value of the median of the medians
5. Partition A with x as the pivot. Let q be the index of x after the partitioning
6. If $q = k$ then return $A[q]$
7. If $k < q$ return **MM-Select**($A[1, \dots, q - 1], k$)
else return **MM-Select**($A[q + 1, \dots, n], k - q$)

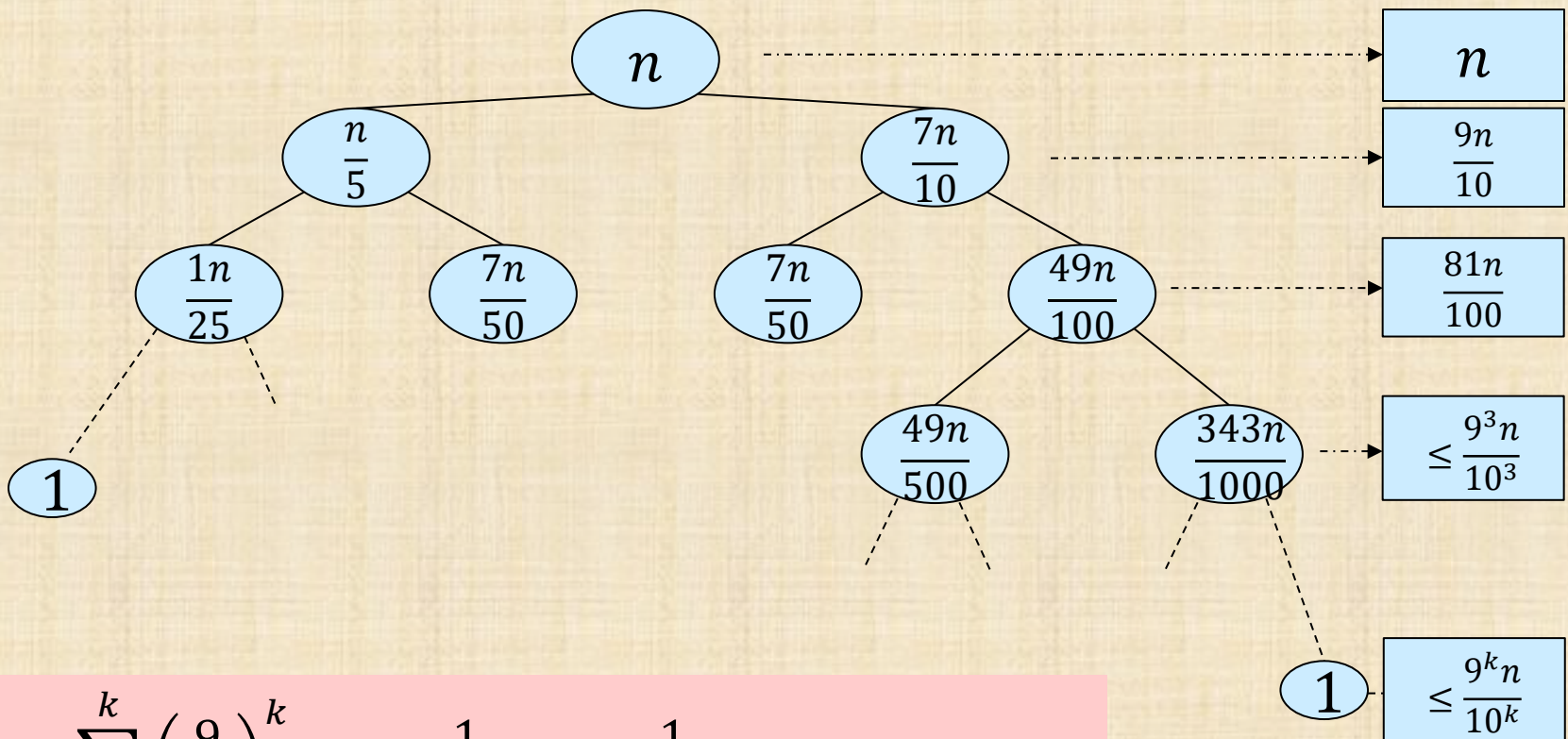
MM-Select(A,k)

1. If $n < 10$, compute the median naively and return it $\Theta(1)$
2. Divide the n elements into $n/5$ groups S_i of 5 elements each. $\Theta(n)$
3. Find the median s_i of each group S_i $n/5 \cdot \Theta(1) = \Theta(n)$
4. Recursively call **MM-Select**($\{s_1, \dots, s_{n/5}\}, \frac{n}{10}$) to find x , the value of the median of the medians $T(n/5)$
5. Partition A with x as the pivot. Let q be the index of x after the partitioning $\Theta(n)$
6. If $q = k$ then return $A[q]$
7. If $k < q$ return **MM-Select**($A[1, \dots, q - 1], k$)
else return **MM-Select**($A[q + 1, \dots, n], k - q$) $T(7n/10)$

$$T(n) \leq \begin{cases} T(n/5) + T(7n/10) + \Theta(n) & \text{if } n \geq 10 \\ \Theta(1) & \text{if } n < 10 \end{cases}$$

Proving $T(n) = O(n)$

$$T(n) \leq \begin{cases} T(n/5) + T(7n/10) + dn & \text{if } n \geq 10 \\ b & \text{if } n < 10 \end{cases}$$



$$n \sum_{i=0}^k \left(\frac{9}{10}\right)^i \leq n \frac{1}{1 - \frac{9}{10}} = n \frac{1}{\frac{1}{10}} = 10n \in O(n)$$

$$n \sum_{i=0}^k \left(\frac{9}{10}\right)^i$$

Median of Medians

- ▷ Time complexity:
 - ▷ Best / worst / average $\Theta(n)$
- ▷ Note: the constant of the $\Theta(n)$ is large.
The algorithm is not fast in practice.

Summary

- ▷ Linear sorting algorithms
- ▷ Order statistics

Next :

- ▷ Binary search trees